APUNTES DE COMPILADORES E INTÉRPRETES

29 DE OCTUBRE

Kevin Jiménez Escalante (2024253395)

DE CFG A TABLA

Anteriormente se había visto cómo usar una tabla de parsing para reconocer el lenguaje generado por una CFG, pero ¿cómo se construyen?

CFG → TABLA
- Preproceso
- Calcular FIRST
- Calcular FOLLOW
- Calcular PREDICT
- Construir la tabla

PREPROCESO
RECURSIVIDAD IZQUIERDA

Anteriormente se había corregido la recursividad a la izquierda, puesto que impedía el funcionamiento de los algoritmos para convertir una CFG a una tabla.

Si tenemos reglas de la forma:
A → Aa
A → b

Se puede arreglar con una regla adicional
A → bA’
A’ → aA’
A’ → ε

PREPROCESO
FACTORIZACIÓN IZQUIERDA

Otra situación problemática puede ocurrir si tenemos dos o más reglas de la forma:

$A \rightarrow ab$

$A \rightarrow ac$

Se puede arreglar con una regla adicional:

$A \rightarrow aA'$

$A' \rightarrow b$

$A' \rightarrow c$

PREPROCESO
FACTORIZACIÓN IZQUIERDA

Nótese que “b” o “c” pueden ser ε:
A → BC
A → B

Que se arregla de la siguiente manera:
A → BA’
A’ → C
A’ → ε

LA FUNCIÓN FIRST

Sea G una CFG. La función FIRST(X) regresa un conjunto de terminales y no terminales (puede regresar ε) que indica todos los inicios “factibles” de X bajo la gramática G.

X puede ser un terminal, no terminal o una hilera (puede ser ε).

El cálculo de FIRST es iterativo y se termina cuando no hay cambios.



Definitivamente no es la primera imagen que me apareció en el buscador cuando busqué “first”.

COMO CALCULAR FIRST(X)

Casos triviales:
- Si X es ε, regresa el conjunto {ε}
- Si X es un terminal a, regresa {a}

Si X es un no terminal, en cada regla de la forma X → ABC...
- Unir (FIRST(A)-{ε}) a FIRST(X)
- Si FIRST(A) contiene a ε hay que unir (FIRST(B)-{ε})
- Si FIRST(B) contiene a ε hay que unir (FIRST(B)-{ε})...

Si X es una hilera ABC...
- Unir (FIRST(A)-{ε}) a FIRST(X)
- Si FIRST(A) contiene a ε hay que unir (FIRST(B)-{ε})
- Si FIRST(B) contiene a ε hay que unir (FIRST(B)-{ε})...

La verdad que es medio despiche, espero que el ejemplo ayude...

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅		
OP	∅		
T	∅		
M	∅		
F	∅		

En la primera regla, hay que unir (FIRST(OP) - {ε}) a FIRST(E), pero FIRST(OP) está vacío entonces se queda igual.

Lo mismo pasa en la segunda regla, no cambia nada.



Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅		
OP	{+,-}		
T	∅		
M	∅		
F	∅		

En la tercera y cuarta regla, se une {+} y {-} a FIRST(OP), por lo que FIRST(OP) = {+,-}.

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅		
OP	{+,-}		
T	∅		
M	∅		
F	∅		

Al igual que con la primera y segunda regla, estamos uniendo los no terminales M y F, que de momento están vacíos.

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅		
OP	{+,-}		
T	∅		
M	{*}		
F	{(,#}		

Estas 3 reglas agregan terminales a FIRST de M y F.

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅	∅	
OP	{+,-}	{+,-}	
T	∅	{(,#}	
M	{*}		
F	{(,#}		

En la segunda pasada no hay cambios hasta la regla 6, donde hay que agregar FIRST(F) a FIRST(T), esta vez no es vacío.

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅	∅	{(,#}
OP	{+,-}	{+,-}	
T	∅	{(,#}	
M	{*}	{*}	
F	{(,#}	{(,#}	

En la tercera pasada hay que agregar FIRST(T) a FIRST(E).

Ejemplo 1:

E → E *OP* T
E → T
OP → +
OP → -
T → T M F
T → F
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅	∅	{(,#}
OP	{+,-}	{+,-}	{+,-}
T	∅	{(,#}	{(,#}
M	{*}	{*}	{*}
F	{(,#}	{(,#}	{(,#}

Después de esto no hay cambios en esta pasada ni en la siguiente, listísimo :D

Ejemplo 2:

S → A B c
A → a
A → ε
B → b
B → ε

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
S	∅		
A	{a,ε}		
B			

Este ejemplo nos va a ayudar a entender mejor cómo trabajar con ε, que es lo más confuso de todo esto.

En la tercera regla encontramos comportamiento nuevo. Primero se agrega (FIRST(ε) - {ε}), por lo que se queda igual.

Pero luego usamos la cláusula que dice que si el FIRST de todos los elementos de la hilera contienen a ε, entonces sí vale agregarlo.

Esto es lo único interesante de esta pasada, entonces revisemos la siguiente.

Ejemplo 2:

S → A B c
A → a
A → ε
B → b
B → ε

NO TERMINAL	PASADA 1	PASADA 2
S	∅	{a,b,c}
A	{a,ε}	
B	{b,ε}	

Ahora sí van a pasar cosas interesantes en la primera regla.

Primero unimos (FIRST(A) - {ε}) a FIRST(S).

Como FIRST(A) contiene a {ε}, también hay que agregar FIRST(B) a FIRST(S).

Pero FIRST(B) también contiene a {ε}, entonces hay que agregar FIRST(c).

Ejemplo 2:

S → A B c
A → a
A → ε
B → b
B → ε

NO TERMINAL	PASADA 1	PASADA 2
S	∅	{a,b,c}
A	{a,ε}	{a,ε}
B	{b,ε}	{b,ε}

El resto queda igual :D

Ejemplo 3:

E → T E’
E’ → OP T E’
E → ε
OP → +
OP → -
T → F T’
T’ → M F T’
T’ → ε
M → *
F → (E)
F → #

NO TERMINAL	PASADA 1	PASADA 2	PASADA 3
E	∅	∅	{(,#}
E’	{ε}	{+,−,ε}	{+,−,ε}
OP	{+,−}	{+,−}	{+,−}
T	∅	{(,#}	{(,#}
T’	{ε}	{ε,*}	{ε,*}
M	{*}	{*}	{*}
F	{(,#}	{(,#}	{(,#}

El ejemplo 1 pero con un par de ε para molestar un poco.

LA FUNCIÓN FOLLOW

Sea G una CFG. La función FOLLOW(X) regresa un conjunto de terminales (puede regresar $) que indica todos las continuaciones “factibles” de X bajo la gramática G.

X siempre va a ser un no terminal.

El cálculo de FOLLOW es iterativo y se termina cuando no hay cambios. FOLLOW nunca contiene ε. Se necesita FIRST para calcular FOLLOW.



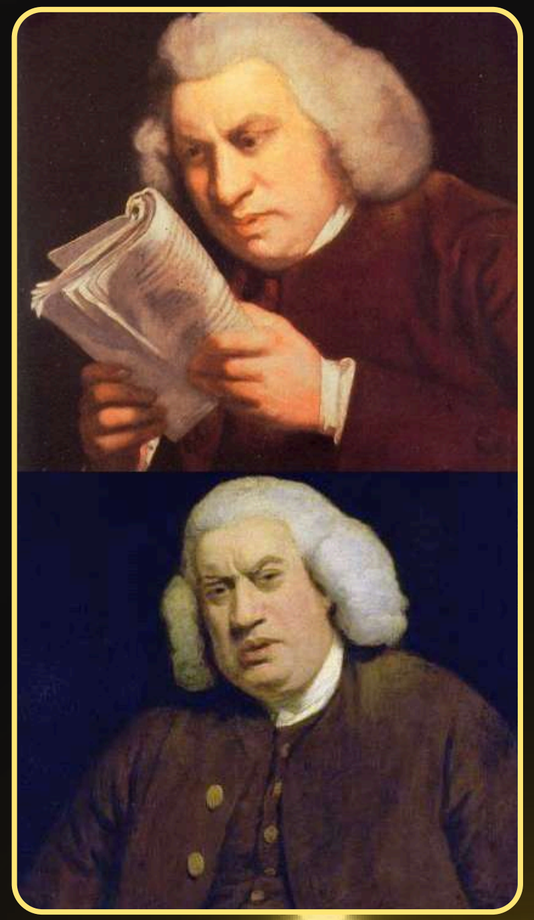
Imágenes stock >> Imágenes hechas con IA

COMO CALCULAR FOLLOW(X)

Casos triviales:
- Si X es el símbolo inicial, agregar $ a FOLLOW(X)
- Se ignoran todas las reglas de la gramática que tengan un sólo terminal a la derecha

Si hay una regla de la forma A → aXb:
- Agregar (FIRST(b) - {ε}) a FOLLOW(X)
- Si FIRST(b) contiene ε, agregar FOLLOW(A) a FOLLOW(X)

La verdad que es medio despiche, espero que el ejemplo ayude...



Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	∅	
OP	{+,−}	∅	
T	{(,#}	∅	
M	{*}	∅	
F	{(,#}	∅	

Primero, se ignoran las reglas que sólo tengan un terminal a la derecha.

Vale la pena poner una columna con el FIRST de cada no terminal en la TABLA porque se va a consultar constantemente.

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,−}	∅	
T	{(,#}	∅	
M	{*}	∅	
F	{(,#}	∅	

Lo primero primerísimo es agregar $ al FOLLOW del símbolo inicial E.

Ahora, hay que agregar (FIRST(OP) - {ε}).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,−}	{(,#}	
T	{(,#}	∅	
M	{*}	∅	
F	{(,#}	∅	

Este procedimiento es diferente al first, puesto que cada regla puede causar múltiples cambios en la tabla en la misma pasada. En este caso también se cambia FOLLOW(OP), agregándole (FIRST(T) - {ε}).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,-}	{(,#}	
T	{(,#}	{$,+,-}	
M	{*}	∅	
F	{(,#}	∅	

Como recordatorio:

Si hay una regla de la forma A → aXb:
- Agregar (FIRST(b) - {ε}) a FOLLOW(X)
- Si FIRST(b) contiene ε, agregar FOLLOW(A) a FOLLOW(X)

La primera regla de la CFG se puede interpretar como:

E → E OP T ε.

Sabemos que FIRST(ε) = {ε}, por lo que hay que agregar FOLLOW(E) a FOLLOW(T).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,-}	{(,#}	
T	{(,#}	{$,+,-}	
M	{*}	∅	
F	{(,#}	∅	

La segunda regla no nos da información adicional.

Lo único bueno de este procedimiento es podemos ignorar varias reglas :D

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,−}	{(,#}	
T	{(,#}	{$,+,−,*}	
M	{*}	{(,#}	
F	{(,#}	∅	

En la quinta regla agregamos (FIRST(M) - {ε}) a FOLLOW(T).

También se agrega (FIRST(F) - {ε}) a FOLLOW(M)).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-}	
OP	{+,−}	{(,#}	
T	{(,#}	{$,+,−,*}	
M	{*}	{(,#}	
F	{(,#}	{$,+,−,*}	

Por último, recordar que se puede interpretar la regla como:

T → T M F ε

Lo que significa que hay que agregar FOLLOW(T) a FOLLOW(F).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-,)}	
OP	{+,−}	{(,#}	
T	{(,#}	{$,+,-,*}	
M	{*}	{(,#}	
F	{(,#}	{$,+,-,*}	

La sexta regla no cambia la tabla.

En la octava regla, hay que agregar (FIRST(❱) - {ε}) a FOLLOW(X)

Terminamos la primera pasada :D

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-,)}	{$,+,-,)}
OP	{+,-}	{(,#}	{(,#}
T	{(,#}	{$,+,-,*}	{$,+,-,*,)}
M	{*}	{(,#}	
F	{(,#}	{$,+,-,*}	

Ahora vamos por la segunda pasada. Anteriormente entendimos la razón por la que hay que agregar FOLLOW(E) a FOLLOW(T), pero como FOLLOW(E) cambió hay que traerse los cambios a FOLLOW(T).

Ejemplo:

E → E *OP* T
E → T
~~*OP* → +~~
~~*OP* → −~~
T → T M F
T → F
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,+,-,)}	{$,+,-,)}
OP	{+,−}	{(,#}	{(,#}
T	{(,#}	{$,+,−,*}	{$,+,−,*,)}
M	{*}	{(,#}	{(,#}
F	{(,#}	{$,+,−,*}	{$,+,−,*,)}

Con el mismo razonamiento se agrega de nuevo FOLLOW(T) a FOLLOW(F).

Después de todo esto hemos terminado :D

Esta es una foto de cómo quedé después de hacer este ejemplo.



Ejemplo 2:

E → T E’
E’ → OP T E’
~~E → ε~~
~~OP → +~~
~~OP → -~~
T → F T’
T’ → M F T’
~~T’ → ε~~
~~M → *~~
F → (E)
~~F → #~~

NO TERMINAL	FIRST(X)	PASADA 1	PASADA 2
E	{(,#}	{$,)}	{$,)}
E’	{-,+,ε}	{$}	{$,)}
OP	{+,-}	{(,#}	{(,#}
T	{(,#}	{$,+,-}	{$,+,-,)}
T‘	{*,ε}	{$,+,-}	{$,+,-,)}
M	{*}	{(,#}	{(,#}
F	{(,#}	{$,+,-,*}	{$,+,-,*,)}

El ejemplo anterior pero con un par de ε para molestar un poco.

GRACIAS POR VER :D